

DASS A3 Task 1

Institute Management System (IMS)

Team Members

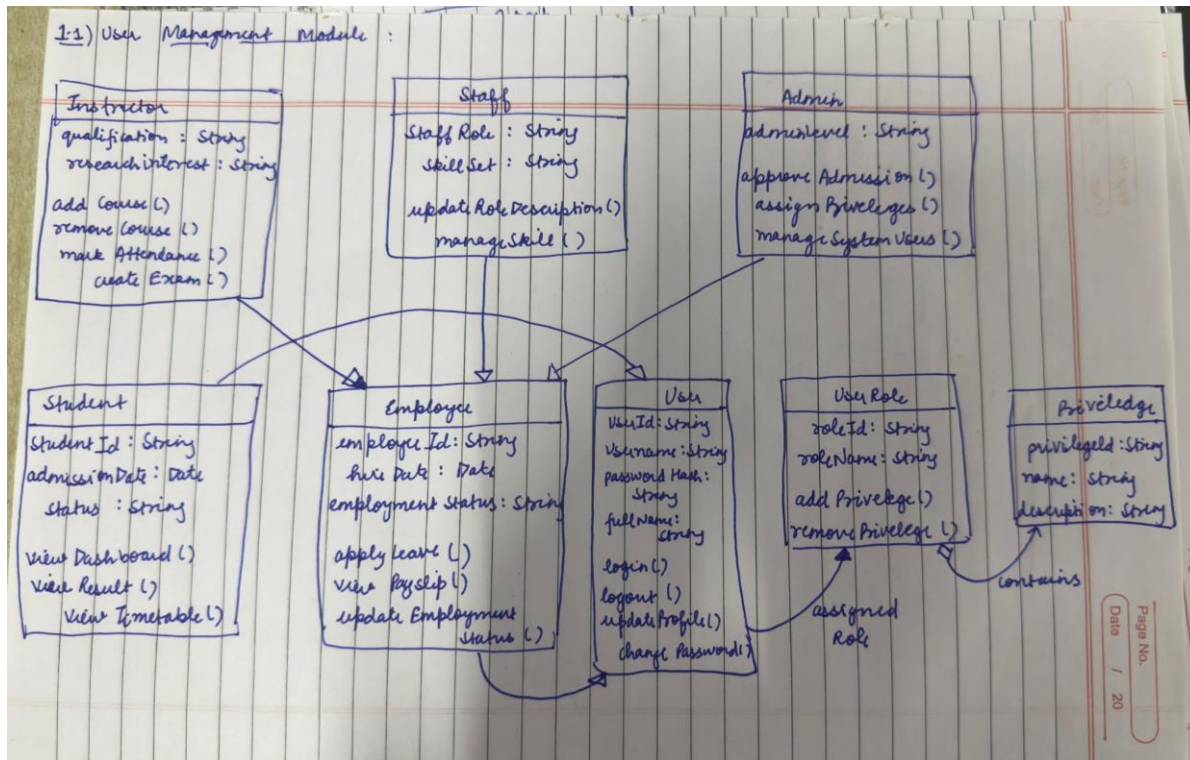
Manik Bansal - 2024101084

Arnav Agnihotri - 2024101103

1. Responsibilities of Each Module

1.1 User Management Module

Defines the system actors, authentication hierarchy, and role-based privilege model.



Class responsibility table

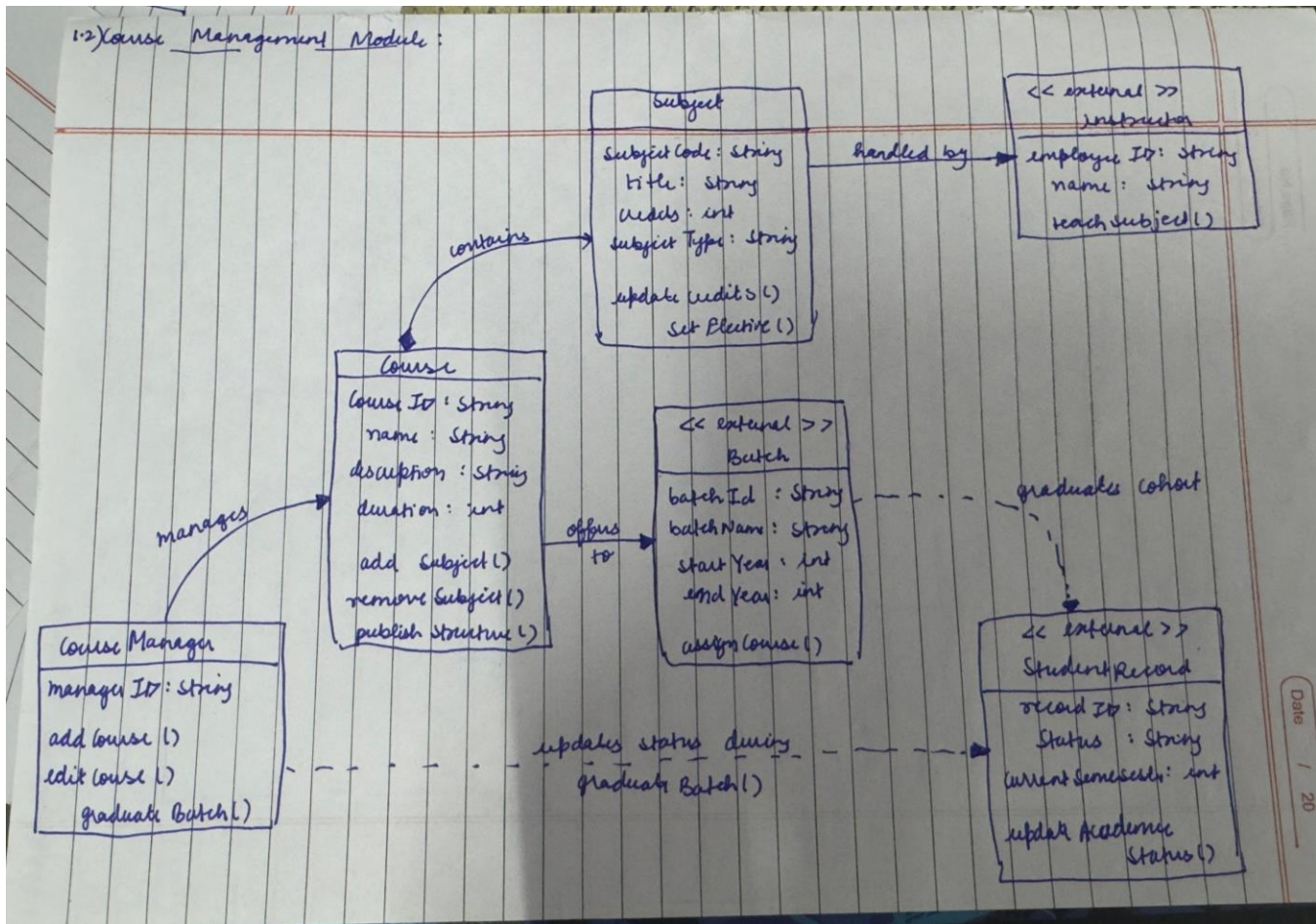
Class	Responsibility	Key attributes	Key operations
User	Abstract base class for all authenticated actors in the IMS. Centralizes credentials, common profile data, and generic account operations, while linking each user to a reusable UserRole for RBAC.	userId, username, passwordHash, fullName	login(), logout(), updateProfile(), changePassword()
Student	Specialized user representing a learner. Adds academic identity and student-specific access paths over the base user profile.	studentId, admissionDate, status	viewDashboard(), viewResult(), viewTimetable()
Employee	Specialized user representing institute employees. Provides the base employee identity used by instructor, staff, and admin roles.	employeeId, hireDate, employmentStatus	applyLeave(), viewPayslip(), updateEmploymentStatus()
Instructor	Teaching employee responsible for subject delivery, attendance	qualification, researchInterest	addCourse(), removeCourse(), markAttendance(),

	workflows, and exam management.		createExam()
Staff	Non-teaching employee role used for operational staff functions such as finance, admin office, or HR support.	staffRole, skillSet	updateRoleDescription(), manageSkill()
Admin	Privileged employee role that supervises the system, approves sensitive actions, and manages permissions.	adminLevel	approveAdmission(), assignPrivileges(), manageSystemUsers()
UserRole	Encapsulates named roles used in RBAC and manages assignment or removal of reusable privileges.	roleId, roleName	addPrivilege(), removePrivilege()
Privilege	Represents one fine-grained permission definition that is assigned through UserRole rather than granting itself.	privilegeId, name, description	-

1.2 Course Management Module

Captures the academic structure that connects courses, subjects, instructors, and batches.

1.2 Course Management Module :



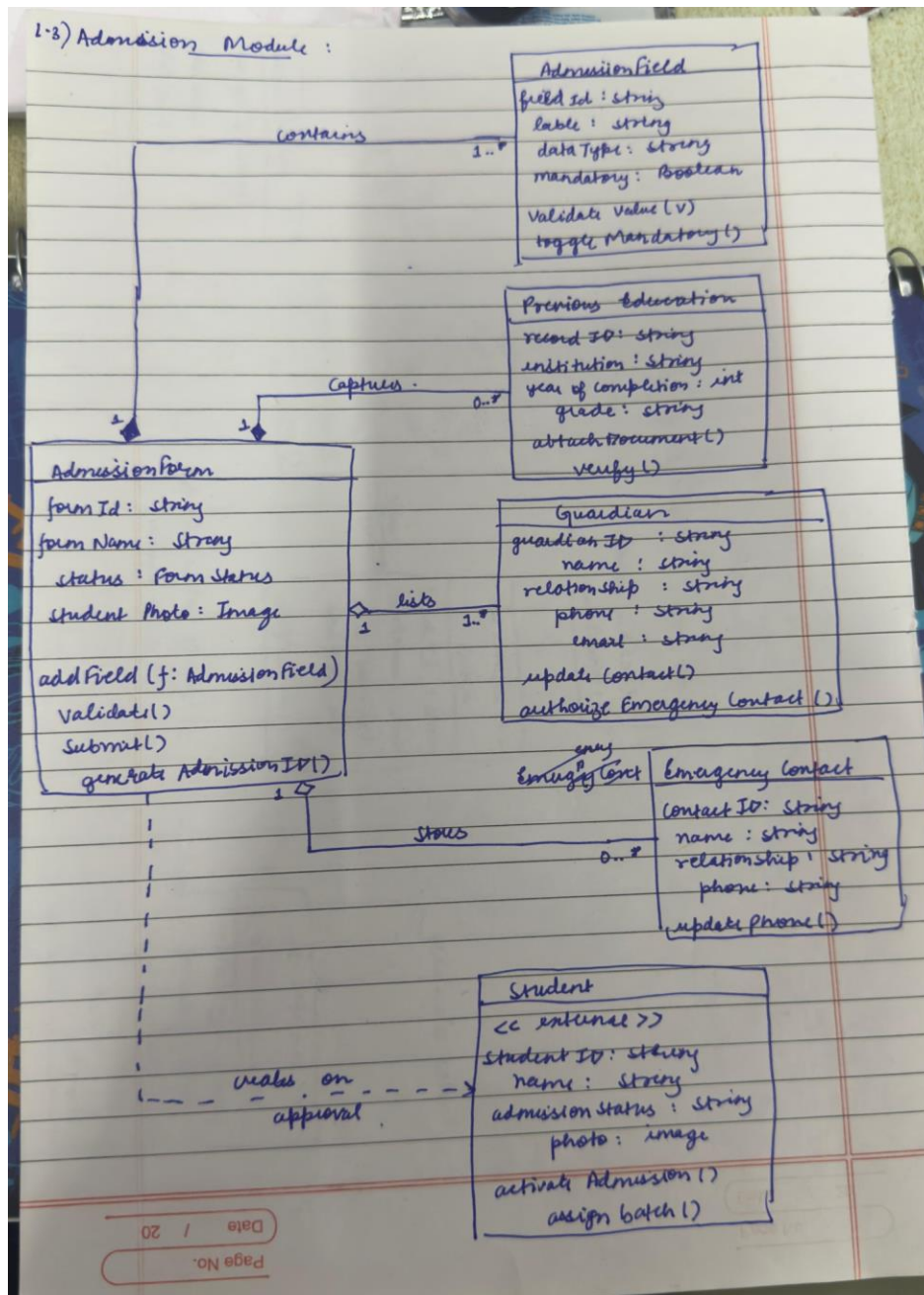
Class responsibility table

Class	Responsibility	Key attributes	Key operations
Course	Defines an academic programme or course of study and acts as the container for its subjects and offered batches.	courseId, name, description, duration	addSubject(), removeSubject(), publishStructure()
Subject	Represents an individual teachable subject/unit with credit value and category such as core or elective.	subjectCode, title, credits, subjectType	updateCredits(), setElective()
CourseManager	Administrative service object responsible for creating, modifying, and maintaining course structures and lifecycle operations. Batch graduation delegates per-student status updates to StudentRecord in the Student Details module.	managerId	addCourse(), editCourse(), graduateBatch()
Batch (external reference)	External shared student cohort associated with a course offering.	batchId, batchName, startYear, endYear	assignCourse()
Instructor	External teaching employee	employeeId, name	teachSubject()

(external reference)	assigned to subjects.		
StudentRecord (external reference)	External shared academic record updated when CourseManager graduates a batch so each student status can be finalized correctly.	recordId, status, currentSemester	updateAcademicStatus()

1.3 Admission Module

Models configurable admission forms and the data required to onboard a new student.

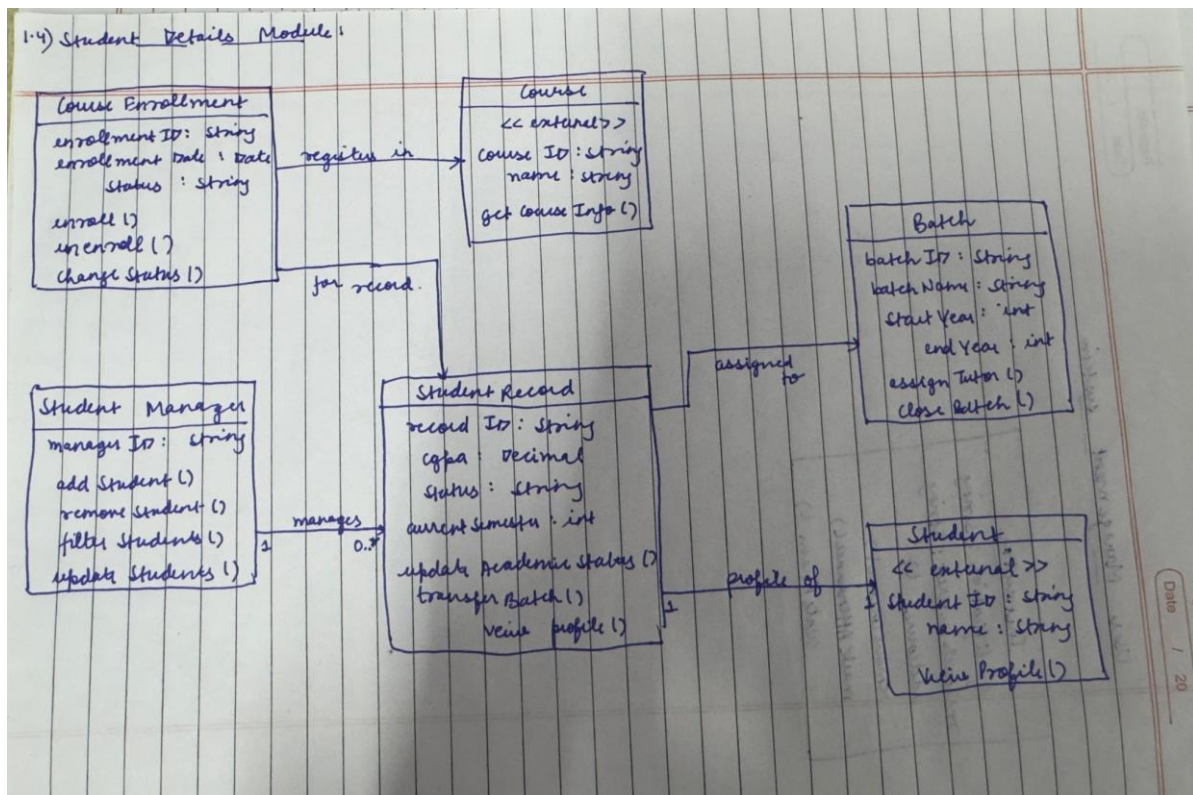


Class responsibility table

Class	Responsibility	Key attributes	Key operations
AdmissionForm	Captures the institute's admission application structure and submitted applicant data. Validates mandatory inputs, supports configurable fields, and initiates creation of a student record after approval.	formId, formName, status, studentPhoto	addField(), validate(), submit(), generateAdmissionId()
AdmissionField	Represents a customizable field inside the admission form. Encapsulates metadata such as field type, label, and whether the field must be completed.	fieldId, label, dataType, mandatory	validateValue(), toggleMandatory()
PreviousEducation	Stores an applicant's prior academic history, including institute, completion year, grade, and supporting documents used during verification.	institution, yearOfCompletion, grade	attachDocument(), verify()
Guardian	Stores guardian information associated with a student applicant. Supports parent/guardian contact tracking required during admission and subsequent communication.	guardianId, name, relationship, phone, email	updateContact(), authorizeEmergencyContact()
EmergencyContact	Maintains fallback emergency contact details for a student, separate from guardian records when necessary.	contactId, name, relationship, phone	updatePhone()
Student (external reference)	External shared entity representing the student created from an approved admission form.	studentId, name, admissionStatus, photo	activateAdmission(), assignBatch()

1.4 Student Details Module

Stores the long-lived academic and administrative profile of enrolled students.

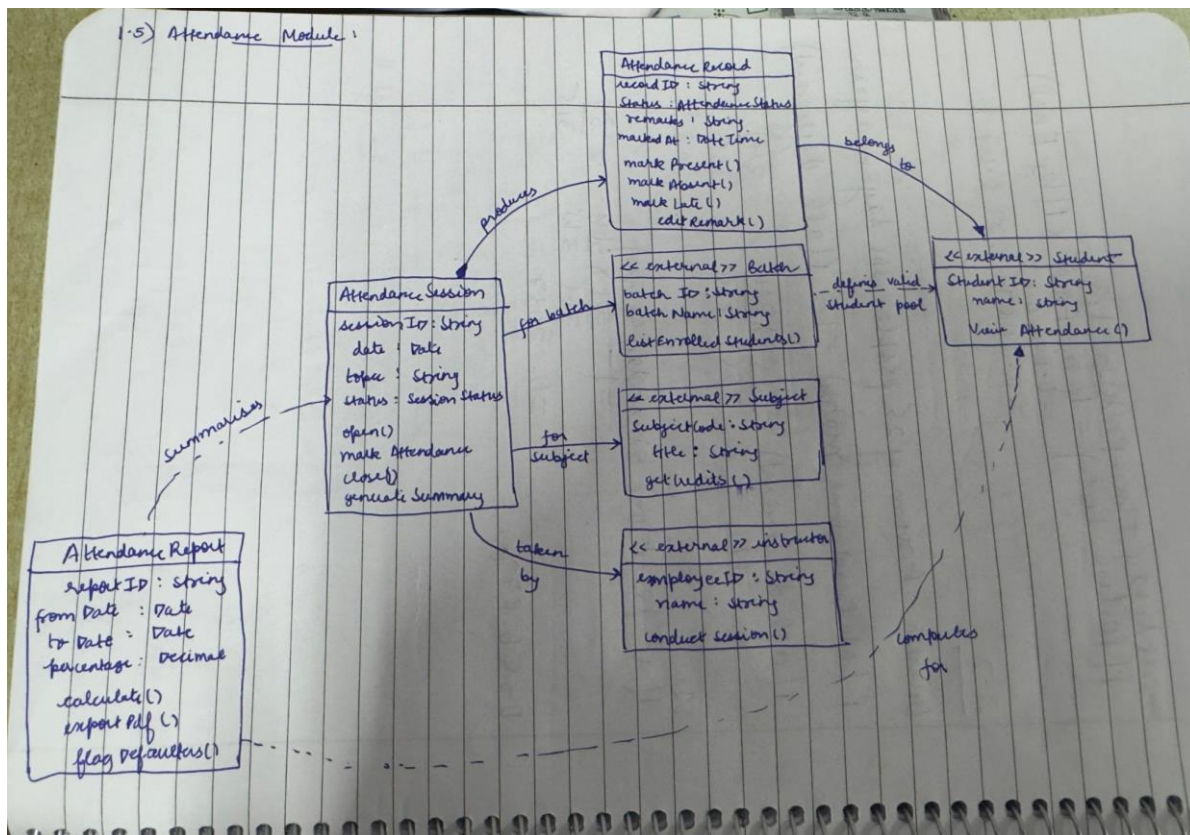


Class responsibility table

Class	Responsibility	Key attributes	Key operations
StudentRecord	Maintains detailed academic and administrative profile information for a student after admission, including status and semester progression.	recordId, cgpa, status, currentSemester	updateAcademicStatus(), transferBatch(), viewProfile()
Batch	Represents a student cohort or section and stores its academic duration and supervision details.	batchId, batchName, startYear, endYear	assignTutor(), closeBatch()
CourseEnrollment	Captures the registration of a student in a course or subject with lifecycle states such as active, dropped, or completed.	enrollmentId, enrollmentDate, status	enroll(), unenroll(), changeStatus()
StudentManager	Provides administrative operations for searching, filtering, and updating student records within the student details module.	managerId	addStudent(), removeStudent(), filterStudents(), updateStudent()
Student (external reference)	External shared student identity linked to the detailed record.	studentId, name	viewProfile()
Course (external reference)	External academic course entity used for registration and progress tracking.	courseId, name	getCourseInfo()

1.5 Attendance Module

Tracks classroom attendance sessions, per-student records, and summary reports.



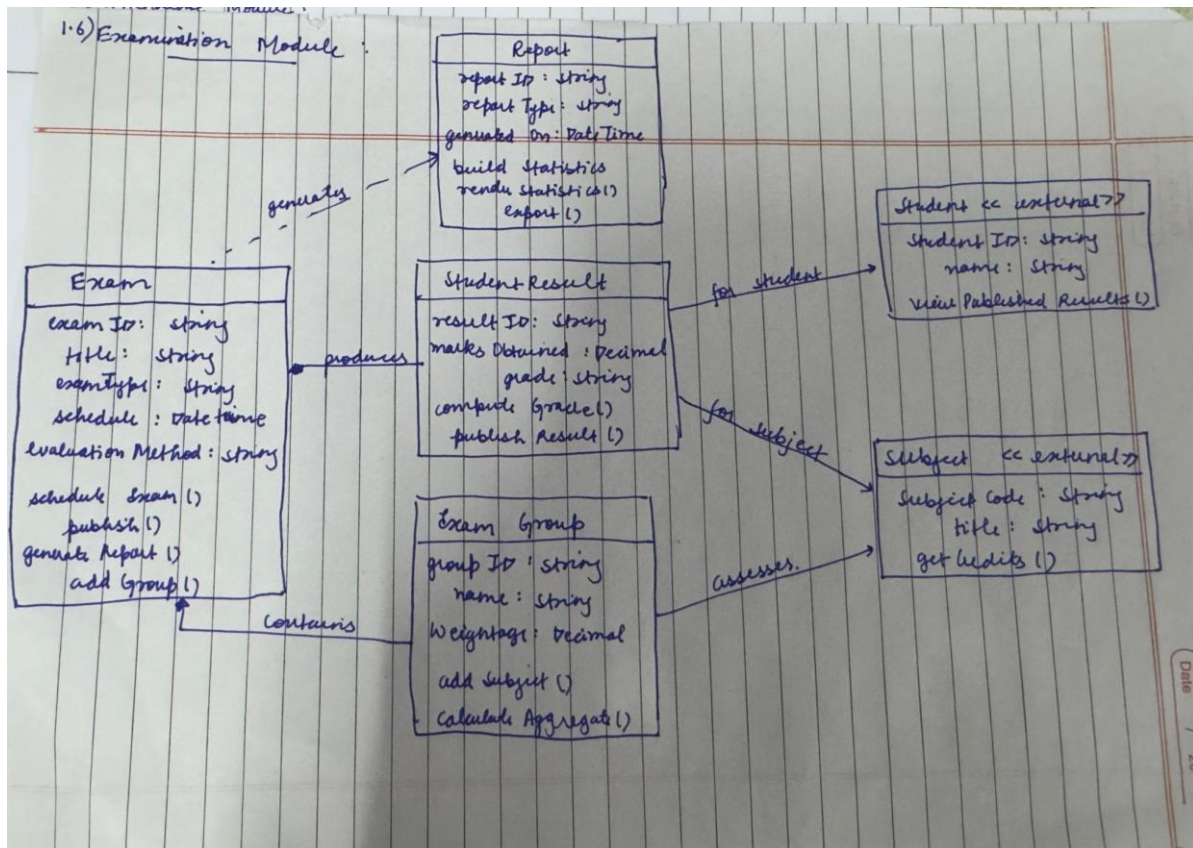
Class responsibility table

Class	Responsibility	Key attributes	Key operations
AttendanceSession	Represents one batch-specific class/lab/tutorial instance for which attendance is taken. Controls the attendance window, validates submissions, and owns the resulting attendance records.	sessionId, date, topic, status	open(), markAttendance(), close(), generateSummary()
AttendanceRecord	Stores one student's attendance outcome for one session, including presence/absence/late status and any supporting remark.	recordId, status, remarks, markedAt	markPresent(), markAbsent(), markLate(), editRemark()
AttendanceReport	Aggregates attendance data across sessions to produce subject-wise or student-wise summaries used for analytics and warning alerts.	reportId, fromDate, toDate, percentage	calculate(), exportPdf(), flagDefaulters()
Student (external reference)	External shared student entity whose attendance is tracked.	studentId, name	viewAttendance()
Subject (external reference)	External academic subject for which the session is conducted.	subjectCode, title	getCredits()
Instructor	External teaching employee responsible for opening and	employeeId, name	conductSession()

(external reference)	closing attendance.		
Batch (external reference)	External shared batch whose enrolled students define the valid attendance pool for a session.	batchId, batchName	listEnrolledStudents()

1.6 Examination Module

Models examinations, grouped assessments, student results, and generated reports.



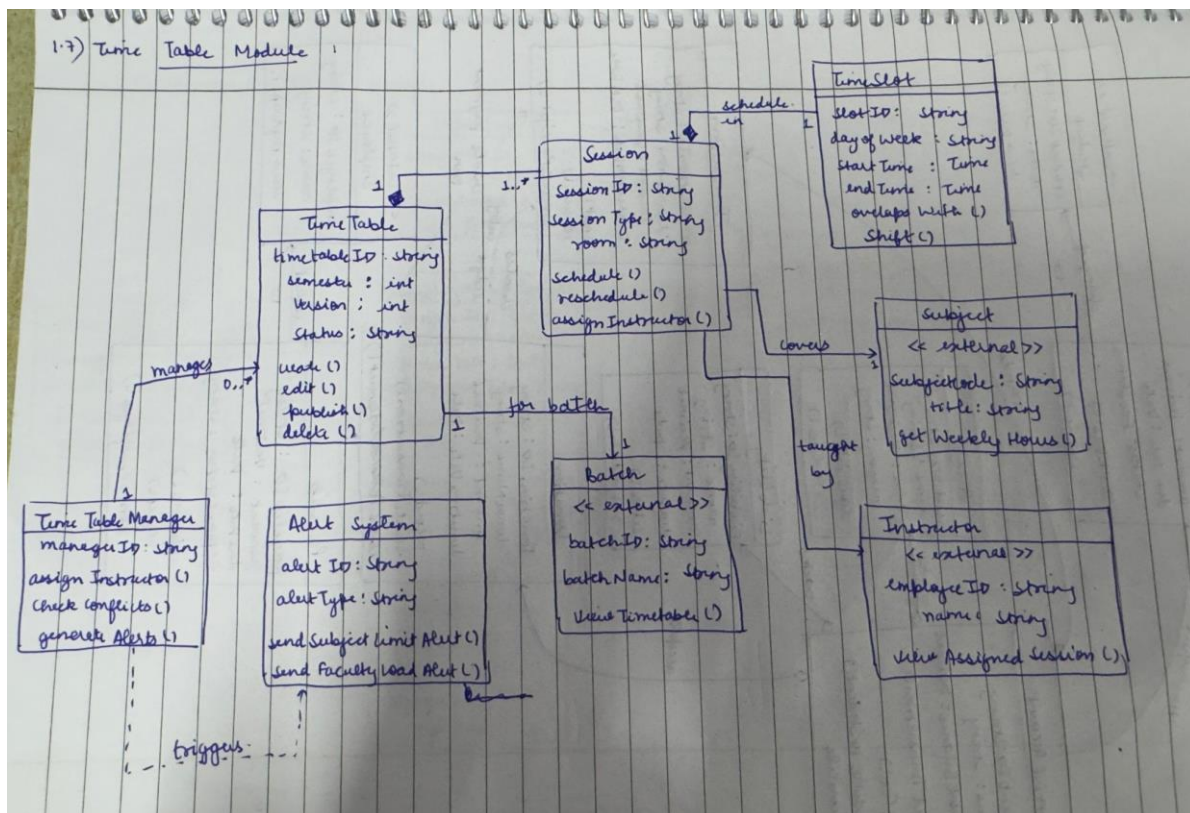
Class responsibility table

Class	Responsibility	Key attributes	Key operations
Exam	Represents a scheduled assessment event. Handles exam metadata, publication, grouping, exam-level evaluation modes, and report generation for marks/grades/custom evaluation schemes.	examId, title, examType, schedule, evaluationMethod	scheduleExam(), publish(), generateReport(), addGroup()
ExamGroup	Groups exam components or subjects under a common evaluation bucket such as theory, lab, or internal	groupId, name, weightage	addSubject(), calculateAggregate()

	assessment, and aggregates linked StudentResult entries when computing group scores.		
StudentResult	Stores a student's scored outcome in an exam, including numeric marks and the derived grade for one student-subject combination.	resultId, marksObtained, grade	computeGrade(), publishResult()
Report	Encapsulates exam analytics output, such as class averages, grade distributions, pass/fail charts, and downloadable result reports.	reportId, reportType, generatedOn	buildStatistics(), renderCharts(), export()
Student (external reference)	External shared student entity receiving examination outcomes.	studentId, name	viewPublishedResults()
Subject (external reference)	External shared subject entity evaluated by the exam.	subjectCode, title	getCredits()

1.7 Time Table Module

Represents scheduling, timeslots, instructor allocation, and timetable alerts.



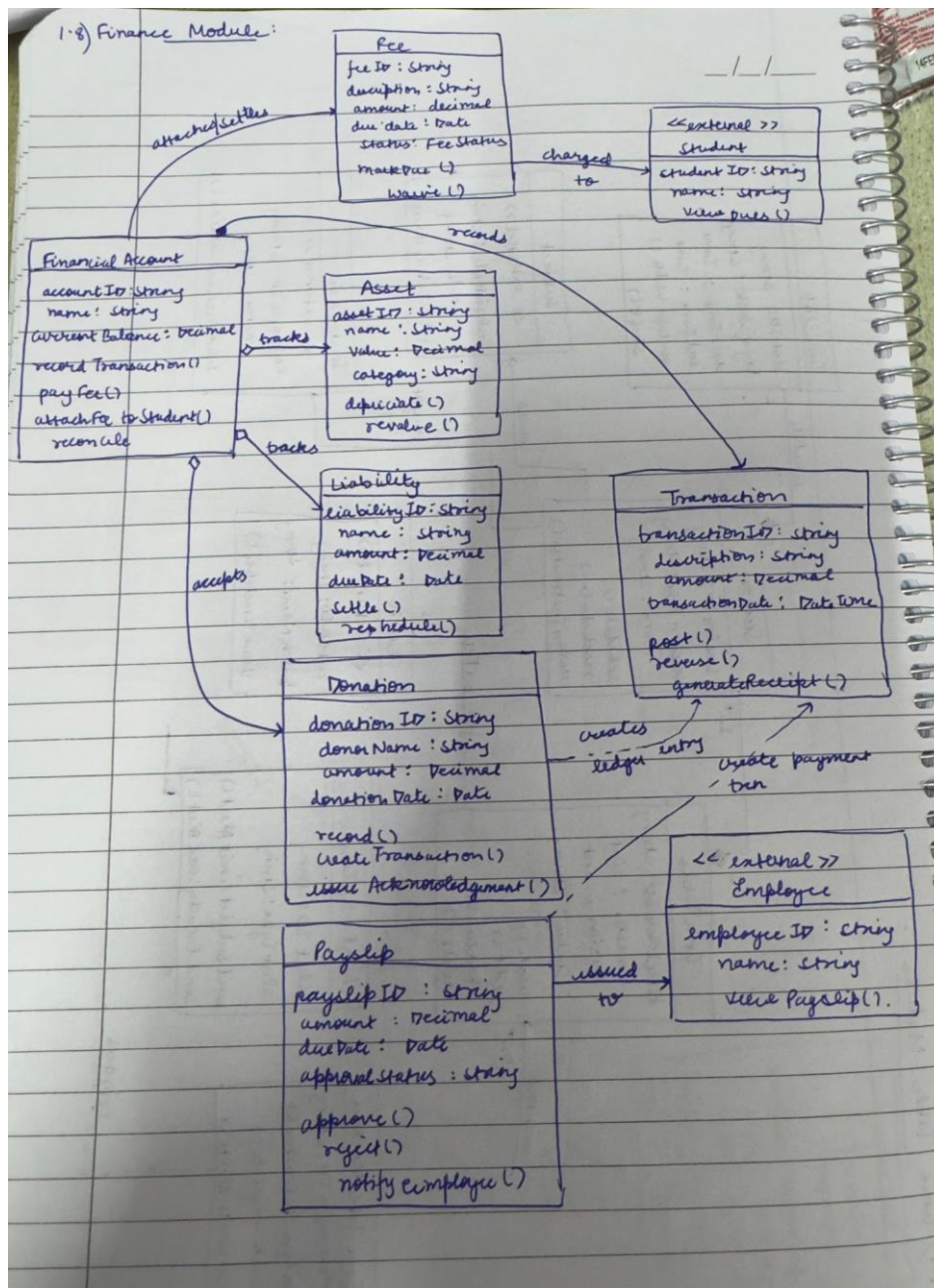
Class responsibility table

Class	Responsibility	Key attributes	Key operations
-------	----------------	----------------	----------------

TimeTable	Acts as the root schedule aggregate for a batch or class group. Supports creation, revision, publication, and version control of the weekly timetable.	timetableId, semester, version, status	create(), edit(), publish(), delete()
Session	Represents a scheduled instructional slot within the timetable for one subject, room, and instructor.	sessionId, sessionType, room	schedule(), reschedule(), assignInstructor()
TimeSlot	Captures the temporal coordinates of a class slot and is reused when validating timetable conflicts.	slotId, dayOfWeek, startTime, endTime	overlapsWith(), shift()
AlertSystem	Generates warnings when timetable design violates subject-hour limits, room constraints, or faculty workload thresholds.	alertId, alertType	sendSubjectLimitAlert(), sendFacultyLoadAlert()
TimeTableManager	Coordinates timetable preparation, conflict detection, instructor allocation, and system alerts for schedulers.	managerId	assignInstructor(), checkConflicts(), generateAlerts()
Subject (external reference)	External shared academic subject scheduled in the timetable.	subjectCode, title	getWeeklyHours()
Instructor (external reference)	External shared instructor allocated to timetable sessions.	employeeId, name	viewAssignedSessions()
Batch (external reference)	External shared batch whose weekly schedule is being built.	batchId, batchName	viewTimetable()

1.8 Finance Module

Covers fee definition, ledger tracking, payroll disbursement, assets, liabilities, and donations.



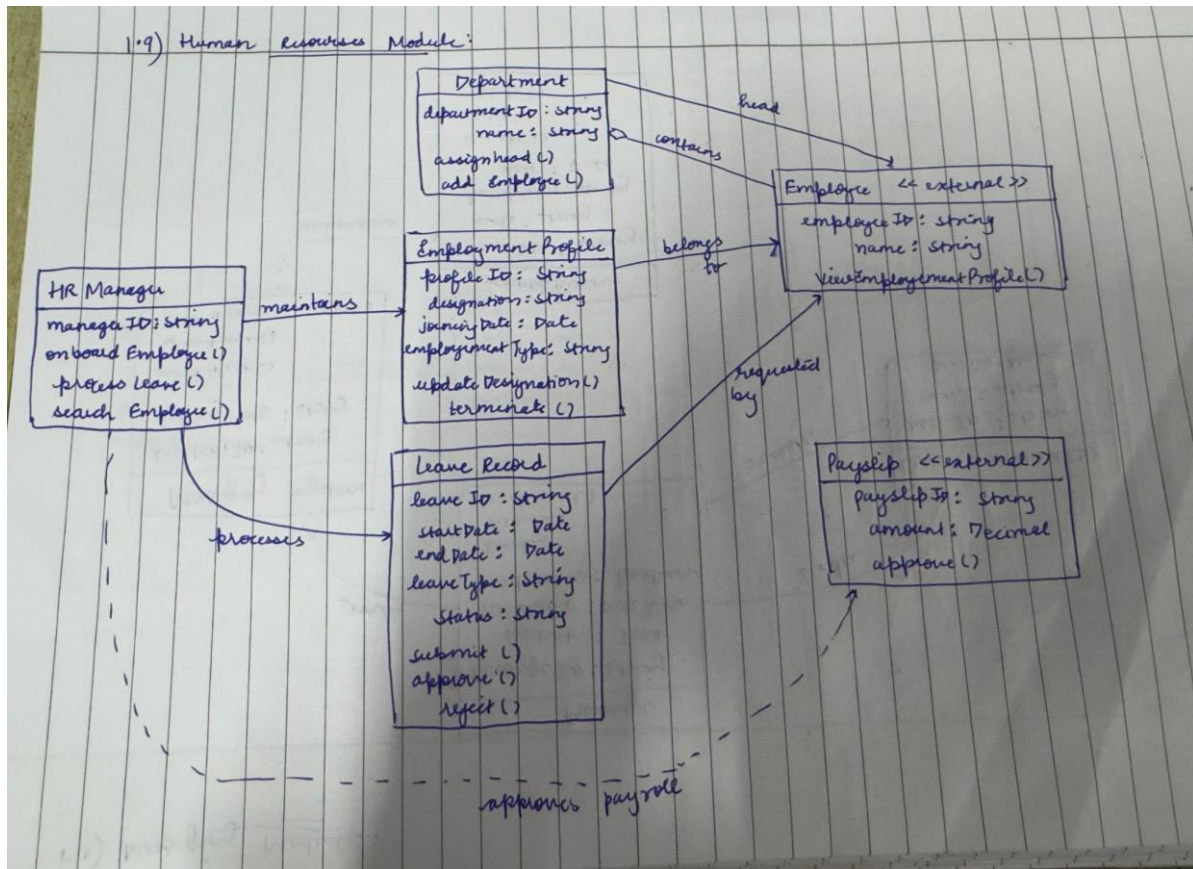
Class responsibility table

Class	Responsibility	Key attributes	Key operations
Fee	Defines an individual fee item to be collected from a student, including amount, due date, and current settlement status. Student attachment is coordinated externally by the account/billing layer.	feeId, description, amount, dueDate, status	markDue(), waive()
FinancialAccount	Maintains account-level balances and acts as the ledger boundary through	accountId, name, currentBalance	recordTransaction(), payFee(), attachFeeToStudent(),

	which finance transactions are recorded and reconciled. It also coordinates fee attachment and donation recording so all money movement reaches the ledger.		recordDonation(), reconcile()
Transaction	Represents an immutable finance ledger event for income, expense, liability settlement, donation receipt, or other account activity.	transactionId, description, amount, transactionDate	post(), reverse(), generateReceipt()
Payslip	Stores payroll payment details prepared for an employee, including payable amount, due date, and approval/rejection workflow.	payslipId, amount, dueDate, approvalStatus	approve(), reject(), notifyEmployee()
Asset	Tracks institute-owned resources with monetary value, such as equipment or infrastructure, for financial reporting purposes.	assetId, name, value, category	depreciate(), revalue()
Liability	Represents a pending financial obligation that must be monitored and settled by the institute.	liabilityId, name, amount, dueDate	settle(), reschedule()
Donation	Captures incoming donation details for institute funding and auditability, and creates a ledger transaction when recorded.	donationId, donorName, amount, donationDate	record(), createTransaction(), issueAcknowledgement())
Student (external reference)	External shared student entity linked to fee collection.	studentId, name	viewDues()
Employee (external reference)	External shared employee entity linked to payslip disbursement.	employeeId, name	viewPayslip()

1.9 Human Resources Module

Manages departments, employee profiles, leave records, and HR processing actions.



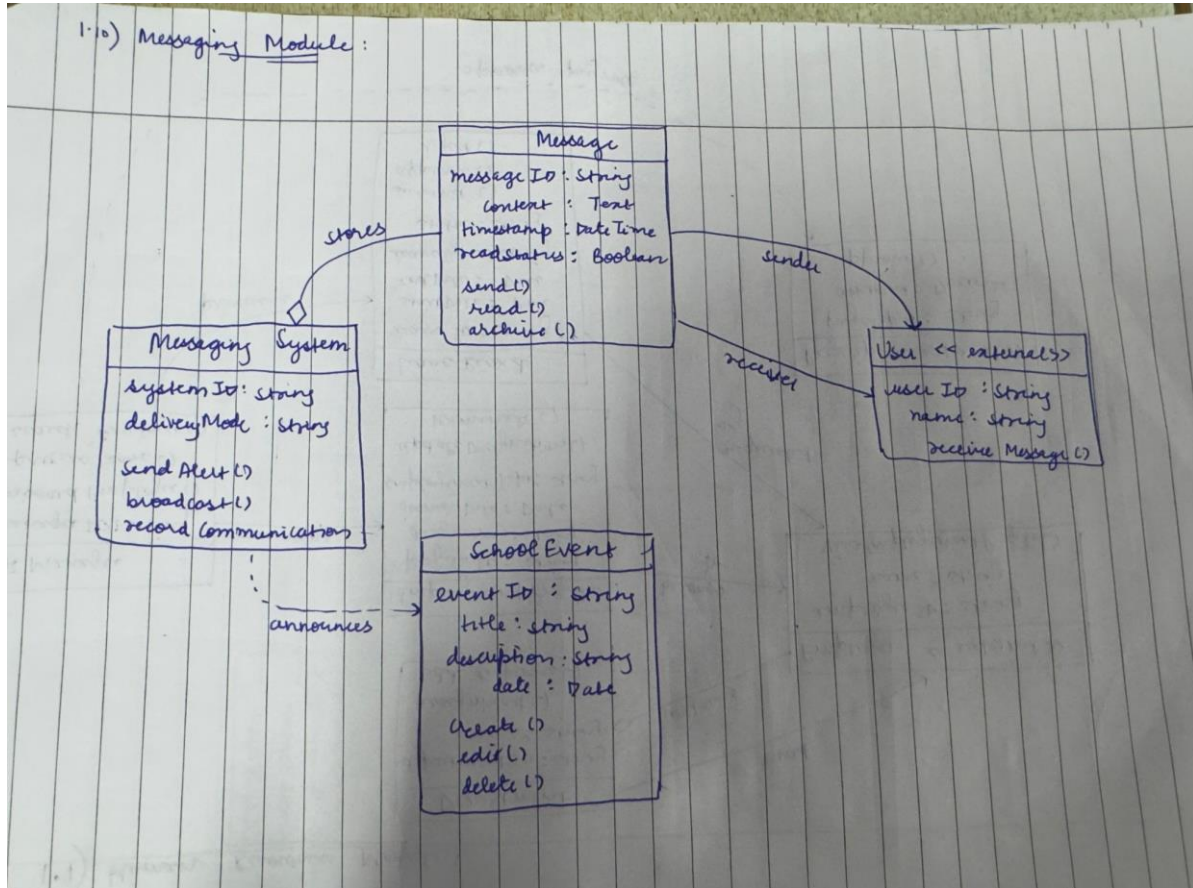
Class responsibility table

Class	Responsibility	Key attributes	Key operations
Department	Represents an organizational unit and groups employees for staffing, reporting, and workload distribution. The department head is modeled through an explicit association to Employee rather than as a free-text field.	departmentId, name	assignHead(), addEmployee()
LeaveRecord	Stores an employee leave request and tracks its lifecycle from submission through approval or rejection.	leaveId, startDate, endDate, leaveType, status	submit(), approve(), reject()
EmploymentProfile	Stores long-lived employment metadata separate from login details, including designation and contract type.	profileId, designation, joiningDate, employmentType	updateDesignation(), terminate()
HRManager	Coordinates HR workflows such as onboarding, leave processing, and employee search/administration.	managerId	onboardEmployee(), processLeave(), searchEmployee()
Employee (external reference)	External shared employee entity managed by HR.	employeeId, name	viewEmploymentProfile()

Payslip (external reference)	External finance payroll artifact referenced by HR during payroll approval flows.	payslipId, amount	approve()
---	---	-------------------	-----------

1.10 Messaging Module

Handles direct messages, alerts, and event-based institute communication.



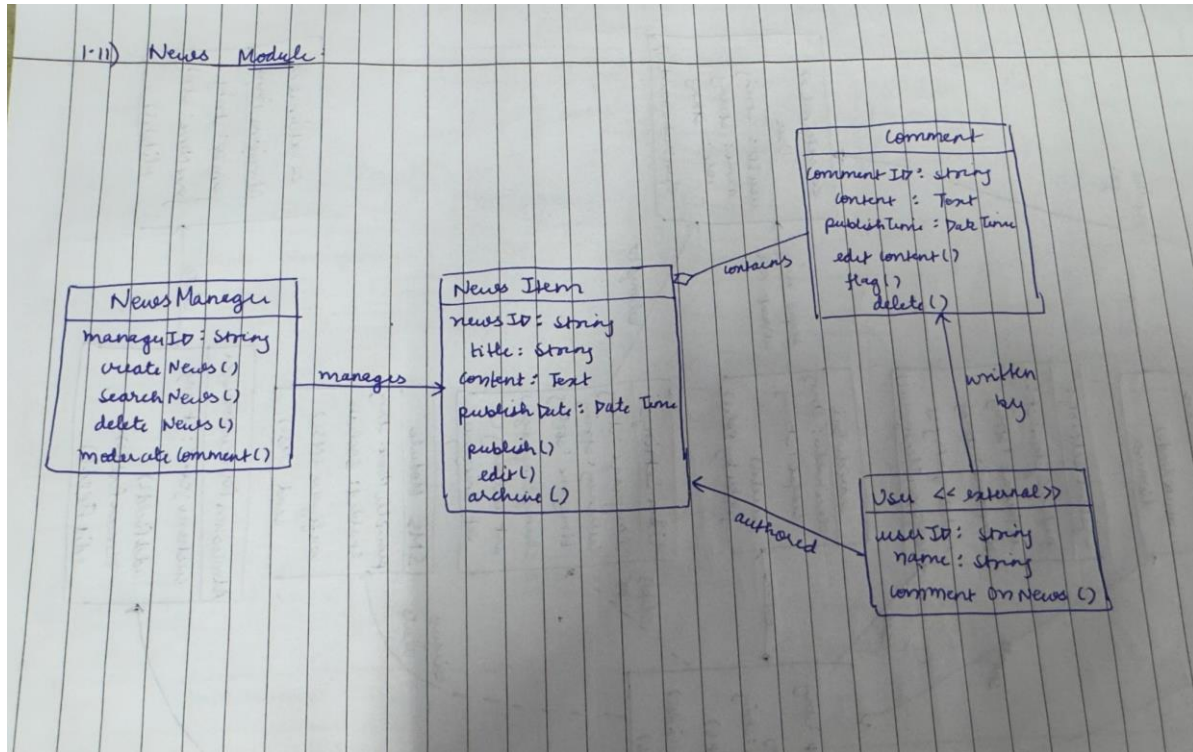
Class responsibility table

Class	Responsibility	Key attributes	Key operations
Message	Represents a direct or targeted message exchanged between institute users with explicit sender and receiver associations, timestamp, and read-state tracking.	messageId, content, timestamp, readStatus	send(), read(), archive()
MessagingSystem	Coordinates message delivery, school alerts, and communication logging across admins, teachers, students, and parents.	systemId, deliveryMode	sendAlert(), broadcast(), recordCommunication()
SchoolEvent	Stores event information that can be broadcast through the messaging subsystem to relevant stakeholders.	eventId, title, description, date	create(), edit(), delete()

User (external reference)	External shared user participating as sender or receiver in the messaging module.	userId, name	receiveMessage()
--------------------------------------	---	--------------	------------------

1.11 News Module

Supports publishing institute news, searching articles, and moderating comments.

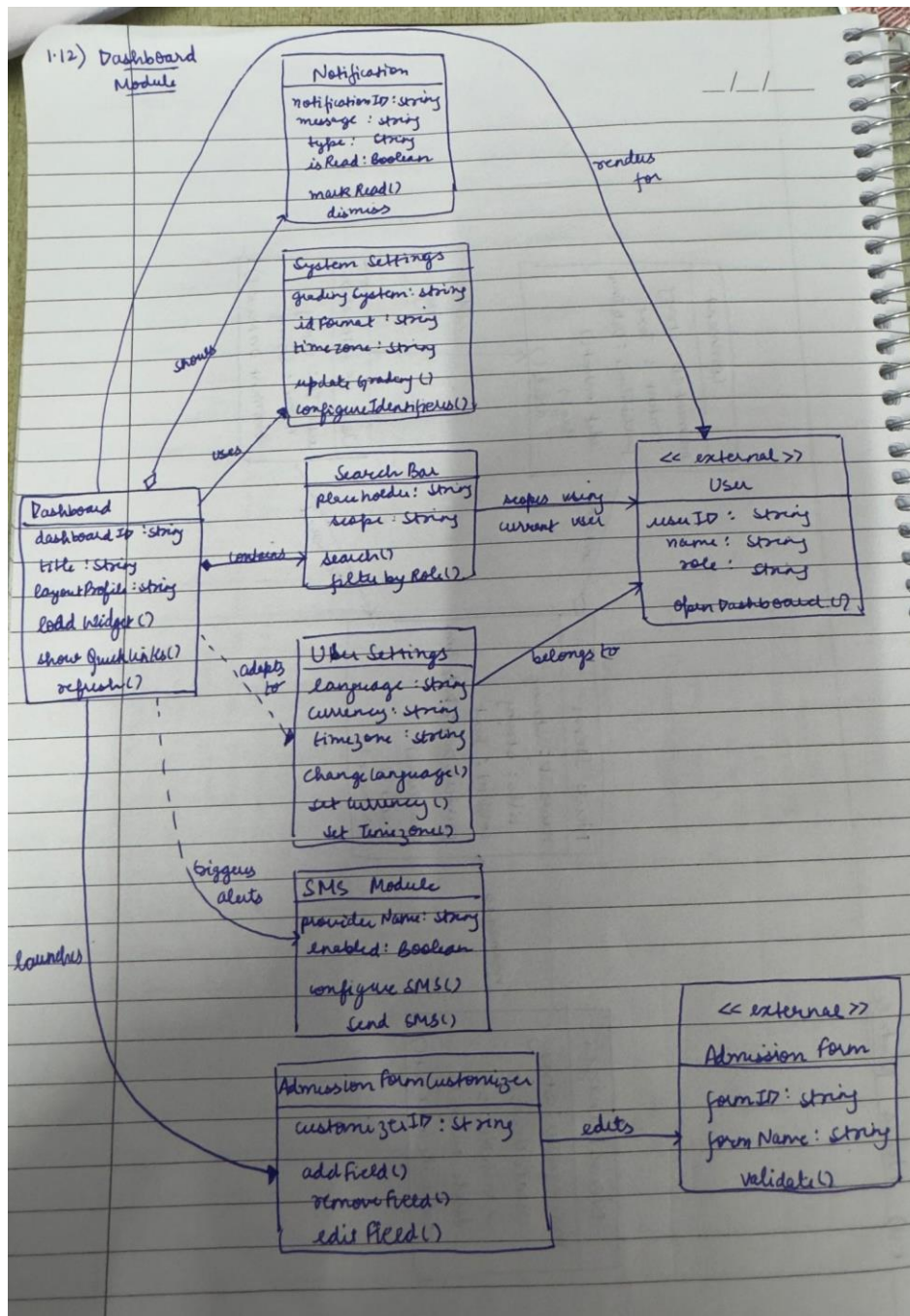


Class responsibility table

Class	Responsibility	Key attributes	Key operations
NewsItem	Represents a rich-text news article published by the institute for announcements, updates, or notices.	newsId, title, content, publishDate	publish(), edit(), archive()
Comment	Stores user feedback or discussion on a published news item and supports moderation actions. Authorship is modeled through an association to User instead of a redundant author-name string.	commentId, content, publishTime	editContent(), flag(), delete()
NewsManager	Administers the news repository, including creation, search, update, deletion, publication status, and comment moderation.	managerId	createNews(), searchNews(), deleteNews(), moderateComment()
User (external reference)	External shared user who authors or comments on news.	userId, name	commentOnNews()

1.12 Dashboard Module

Aggregates navigation, notifications, search, settings, and admission customization helpers.



Class responsibility table

Class	Responsibility	Key attributes	Key operations
Dashboard	Role-aware landing page that aggregates system information, navigation, alerts, and shortcuts into one unified interface.	dashboardId, title, layoutProfile	loadWidgets(), showQuickLinks(), refresh()
SearchBar	Provides instant entity search from the	placeholder, scope	search(), filterByRole()

	dashboard and scopes results according to the current authenticated user's role and permissions.		
Notification	Represents in-app alerts surfaced on the dashboard, such as fee dues, low attendance, or exam result publication.	notificationId, message, type, isRead	markRead(), dismiss()
SystemSettings	Stores global platform configuration such as grading scheme, identifier policies, and institute-level defaults. Dashboard uses these settings but does not own their lifecycle.	gradingSystem, idFormat, timeZone	updateGrading(), configureIdentifiers()
UserSettings	Stores user-specific preferences used when rendering dashboard content and localized interface details for one authenticated user.	language, currency, timeZone	changeLanguage(), setCurrency(), setTimeZone()
AdmissionFormCustomizer	Administrative helper used from the dashboard to customize admission forms and related field definitions.	customizerId	addField(), removeField(), editField()
SMSModule	Provides SMS integration used for dashboard-triggered alerts and broadcast communication.	providerName, enabled	configureSms(), sendSms()
User (external reference)	External shared actor for whom the dashboard is rendered and whose role context also scopes dashboard search.	userId, name	openDashboard()
AdmissionForm (external reference)	External admission entity customized from dashboard admin tools.	formId, formName	validate()

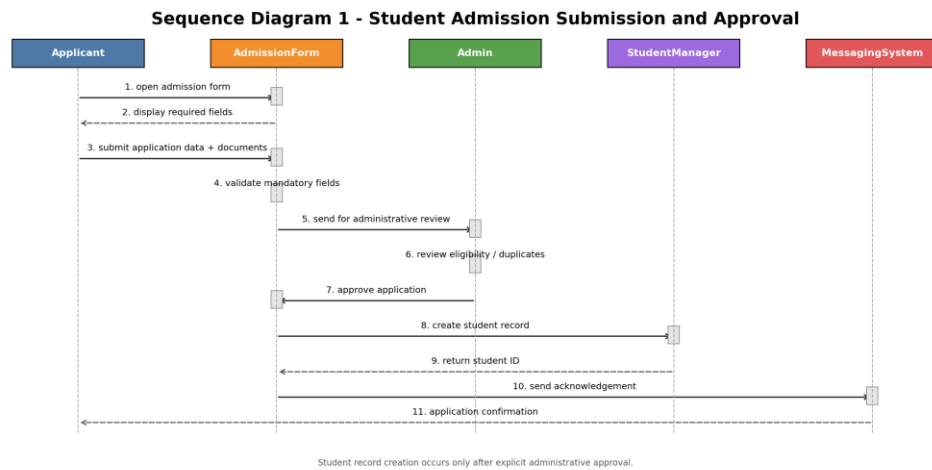


Figure 2.1 – Sequence diagram for student admission submission and approval.

This flow starts when an applicant opens the admission form and ends only after an administrator reviews and approves the submission, the system creates the student record, and MessagingSystem sends an acknowledgement. It demonstrates validation, administrative review, approval, student profile creation, and notification dispatch.

Sequence Diagram 2: Timetable Creation, Validation, and Publication

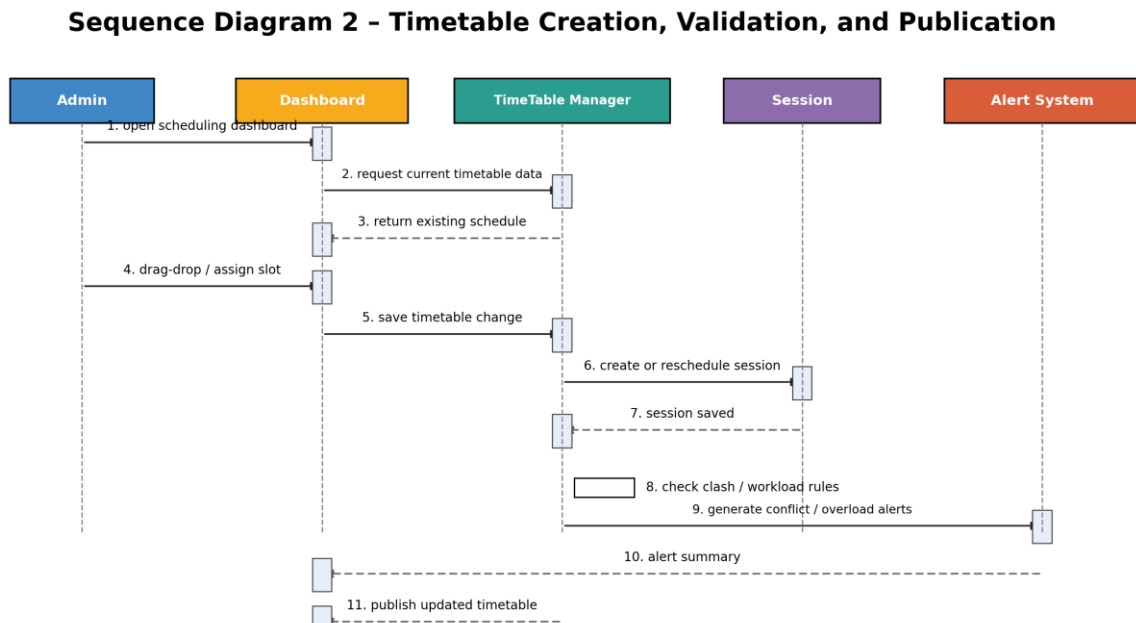


Figure 2.2 – Sequence diagram for timetable creation, validation, and publication.

This flow shows how an administrator uses the dashboard to update the timetable, how the TimeTableManager validates clashes and workload limits, how session objects are created or rescheduled, and how the AlertSystem returns warnings before publication.

Sequence Diagram 3: Examination Evaluation and Result Publication

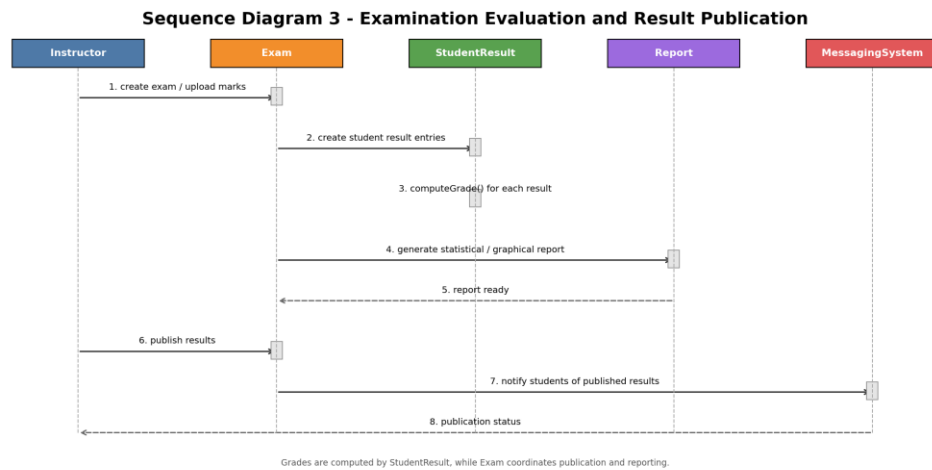


Figure 2.3 – Sequence diagram for examination evaluation and result publication.

This flow covers examination setup, creation of per-student result entries, grade computation inside StudentResult, report generation, and student notification after result publication.

2.2 State Diagram

The state diagram focuses on a single major entity and shows how its valid states and transitions are constrained by business rules.

State Diagram: AdmissionForm Lifecycle

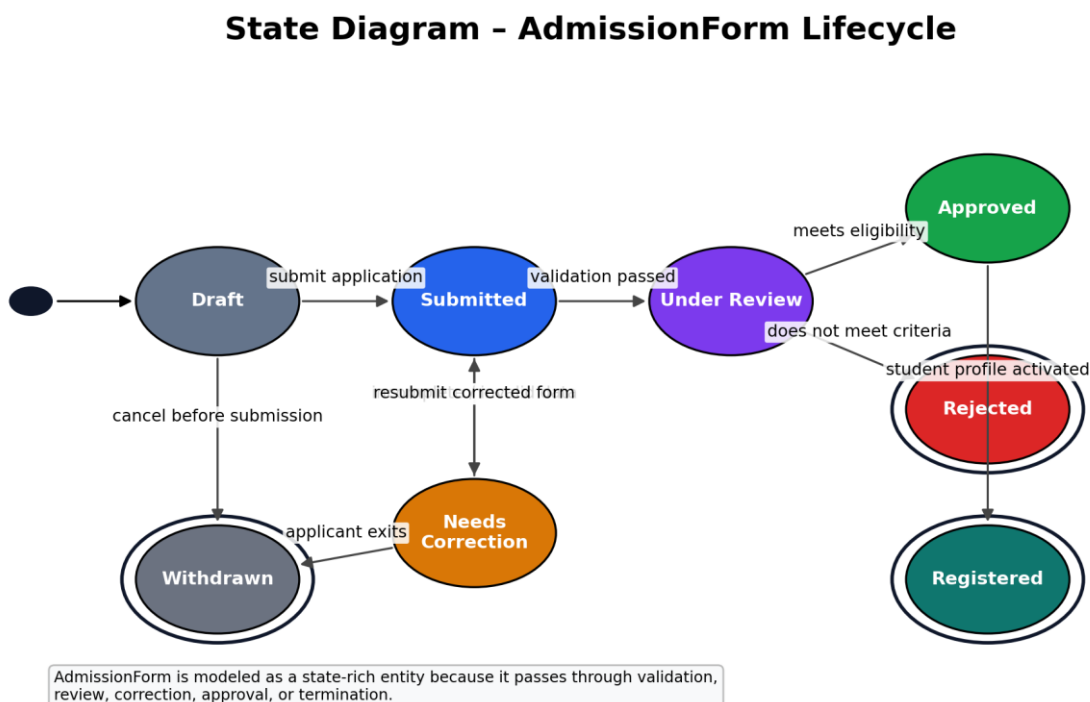


Figure 2.4 – State diagram for the AdmissionForm lifecycle.

AdmissionForm is modeled as a state-rich entity because it moves through draft preparation, formal submission, review, correction, approval, rejection, registration, or withdrawal. This state model prevents invalid transitions and makes the admission workflow explicit.

3. Design Narrative & Analysis

The IMS design balances completeness and maintainability by organizing the system into well-defined modules and by presenting relationships and responsibilities clearly in the underlying IMS design.

3.1 Cohesion

The design emphasizes high cohesion by ensuring that each class has one focused responsibility. For example, AdmissionForm manages only admission data capture and validation-related behavior; TimeTableManager coordinates scheduling logic; StudentResult stores result data and computes grades for one student-subject result entry; and Notification handles alert delivery concerns. Because each class is narrowly scoped, changes in one functional area do not force unrelated changes in other areas.

3.2 Coupling

Coupling is kept intentionally moderate to low. Modules collaborate through a small number of stable entities such as User, Student, Employee, Subject, Batch, and Transaction rather than through many ad hoc references. For instance, the timetable module depends on Subject, Instructor, Session, and AlertSystem, but it does not embed finance or HR rules. Similarly, the examination module references Student and Subject while keeping report generation separate from timetable and admission logic. This structure reduces the ripple effect of change and supports easier testing.

3.3 Separation of Concerns

Separation of concerns is achieved at two levels. First, the system is divided into modules such as Admission, Examination, Time Table, Finance, Human Resources, News, and Dashboard. Second, within a module, entity classes store domain state while manager classes coordinate higher-level operations. For example, NewsItem and Comment represent domain objects, whereas NewsManager coordinates publishing, editing, searching, and moderation. This separation makes the conceptual model easier to understand and keeps operational responsibilities from being scattered across many entities.

3.4 Law of Demeter

The design also follows the Law of Demeter by encouraging classes to interact mainly with their immediate collaborators. Dashboard coordinates access to SearchBar, Notification, SystemSettings, UserSettings, and AdmissionFormCustomizer rather than directly traversing deep object chains across every module. AttendanceSession communicates with AttendanceRecord and the student or subject references it directly owns, and Exam interacts with StudentResult, ExamGroup, and Report without reaching into unrelated internal structures. This reduces unnecessary knowledge sharing between classes and keeps interfaces tighter.

4. Pattern & Anti-Pattern Analysis

The IMS design contains several reusable object-oriented patterns and also deliberately avoids common anti-patterns that would make the system harder to maintain.

4.1 Design Patterns Present

Pattern 1: Façade Pattern in the Dashboard module. Dashboard acts as a unified entry point for multiple services such as search, notifications, admission customization, and user settings. This reduces the number of direct dependencies that the user interface must manage and gives the system a single navigational hub.

Pattern 2: Observer / Event Notification pattern. Events such as admission submission, timetable conflicts, and examination result publication are good candidates for notification-based behavior. Notification, MessagingSystem, and alert-related classes can react to state changes without tightly coupling the initiating module to every possible receiver.

Pattern 3: Manager / Controller coordination pattern. Classes such as StudentManager, TimeTableManager, NewsManager, and CourseManager centralize coordination logic while entities keep their own state. This is consistent with an MVC-style design where managers play a controller-like role for module operations.

4.2 Anti-Patterns Avoided

Anti-pattern 1: God Class. A major risk in an IMS is allowing Dashboard, Admin, or User to absorb too much business logic. The design avoids this by distributing responsibilities across specialized manager and entity classes instead of concentrating admission, finance, scheduling, and examination rules inside a single oversized class.

Anti-pattern 2: Circular Dependency. Cross-module links are present, but the design avoids direct cycles wherever possible. For example, Finance, HR, and User Management share only necessary references such as Employee, Transaction, and Payslip rather than making every module import each other's internal structures.

Anti-pattern 3: Anemic Domain Model. The entities are not treated as passive data bags alone. Classes such as AdmissionForm, Exam, AttendanceSession, and TimeTable own meaningful operations such as submitForm, generateReport, markAttendance, or scheduleSession, which keeps domain behavior close to domain state.

5. Reflection

A short reflection on the strengths and limitations of the proposed IMS design is given below.

5.1 Two Strongest Aspects

- Modular structure with readable package boundaries. Each major concern—admission, timetable, examination, finance, HR, news, messaging, and dashboarding—has a separate conceptual space, making the design easier to understand, extend, and assign to team members.
- The responsibility tables and dynamic diagrams clearly distinguish ownership, lifecycle dependencies, and collaboration among classes

5.2 Two Weakest Aspects

- The overall IMS design remains conceptually dense because the system covers many modules and cross-module references. Even after simplifying the report, the complete design still requires careful inspection.
- Some cross-cutting runtime concerns—such as authorization policy checks, concurrency handling, and notification infrastructure—are only modeled at a conceptual level and would need more detailed design in implementation.

5.3 Suggested Improvement Direction

A strong next step would be to add a more explicit service-layer or interface-layer design for security, persistence, and event handling. That would make the transition from conceptual UML to implementation smoother while preserving the same modular structure already established in Task 1.

6. Modeling Assumptions Used

- User is the abstract parent for all authenticated actors. Student and Employee inherit from User; Instructor, Staff, and Admin inherit from Employee.
- Shared entities such as Student, Employee, Subject, Batch, and User are treated as common concepts across modules even though this refined report presents the system primarily through responsibility tables and dynamic diagrams.
- AdmissionForm creates or activates a Student only after validation and administrative approval.
- AttendanceSession owns AttendanceRecord entries; an attendance record cannot exist independently of its parent session.
- Exam owns StudentResult, groups its assessment parts using ExamGroup, stores evaluationMethod at the Exam level, and represents analytical output through Report. ExamGroup aggregates linked StudentResult values when calculating group scores.
- TimeTable owns Session objects, and each Session must be scheduled in exactly one TimeSlot.
- Fee settlement and payroll approval eventually produce finance ledger entries through Transaction.
- Dashboard is modeled as a coordinating interface layer over notifications, search, settings, and administrative helpers. It uses global SystemSettings, adapts to UserSettings owned by a User, and delegates admission customization through AdmissionFormCustomizer rather than acting as a God class.
- SearchBar scopes results using the currently authenticated User context rather than an isolated free-floating role filter.
- Every AttendanceSession belongs to exactly one Batch so the valid student pool for marking attendance is well-defined.
- Donation recording always produces a Transaction entry to preserve ledger auditability and reconciliation.